DOCUMENTACIÓN TÉCNICA DE SISTEMAS EN PRODUCCIÓN

Proyecto – API de Predicción (Machine Learning + FastAPI + Docker)

1. Introducción

El presente documento describe de forma detallada el diseño, arquitectura e implementación de un sistema de predicción basado en técnicas de Machine Learning, expuesto mediante una API REST de alto rendimiento. En la ingeniería de software actual, el despliegue de modelos predictivos se ha convertido en una pieza fundamental para la transformación digital, exigiendo que los artefactos matemáticos generados por científicos de datos dejen de ser scripts estáticos y pasen a operar como servicios dinámicos y resilientes.

Este proyecto tiene como objetivo principal servir como base de arquitectura empresarial para el desarrollo de aplicaciones backend que integran modelos de inteligencia artificial dentro de sistemas reales, simulando fielmente las restricciones, patrones y directrices de un entorno profesional de desarrollo. A lo largo del ciclo de vida de este sistema, se prioriza el aislamiento de dependencias, la tipificación estricta de datos y el desacoplamiento modular.

El sistema se centra específicamente en la construcción de una interfaz de programación de aplicaciones (API) capaz de recibir datos estructurados en formato JSON por parte de clientes concurrentes, someterlos a reglas estrictas de validación de esquemas, procesarlos en tiempo real mediante un algoritmo de aprendizaje automático previamente entrenado y optimizado, y finalmente devolver una predicción probabilística bajo un formato unificado y estandarizado internacionalmente.

2. Contexto y problema

En los entornos empresariales modernos, la acumulación masiva de datos ha abierto las puertas a la automatización predictiva. La necesidad de tomar decisiones estratégicas basadas en datos en tiempo real es una constante transversal a múltiples sectores de negocio. Entre los ejemplos de uso más comunes, críticos y de alto valor financiero dentro de las organizaciones, se encuentran los siguientes escenarios:

- **Predicción de abandono de clientes (churn):** Permite identificar patrones sutiles de comportamiento y navegación de los usuarios antes de que cancelen sus suscripciones o contratos, habilitando al equipo de marketing a lanzar campañas de retención quirúrgicas.
- **Evaluación de riesgo financiero:** Automatiza y optimiza la evaluación del perfil de riesgo de solicitantes de créditos analizando variables financieras históricas, reduciendo drásticamente la tasa de impagos y agilizando las mesas de aprobación.
- **Estimación de demanda o ventas:** Facilita la planificación logística y de inventarios al predecir los picos de consumo de bienes y servicios, mitigando los costes derivados de quiebres de stock o sobrealmacenamiento.

En este contexto, el principal reto tecnológico de las compañías no radica únicamente en construir el modelo de Machine Learning en entornos experimentales cerrados como Jupyter Notebooks, sino en integrarlo de manera armónica, segura y eficiente dentro de un sistema de software accesible, reutilizable y tolerante a fallos de infraestructura.

Los scripts experimentales suelen carecer de manejo de concurrencia, validación de tipos ante entradas corrompidas de red y documentación estandarizada. Este proyecto aborda precisamente ese problema estructural de raíz: detalla la metodología y la ingeniería necesarias para convertir un modelo matemático en un servicio desacoplado, seguro y altamente consumible mediante una API REST industrial.

3. Objetivos del sistema

Objetivo general

Desarrollar, estructurar e implementar un sistema backend robusto y modular de nivel profesional, capaz de exponer un modelo de Machine Learning clásico como un servicio web accesible a través del protocolo HTTP, garantizando su reproducibilidad mutacional y su estabilidad operativa en entornos de producción cloud.

Objetivos específicos

- **Implementar un modelo de predicción utilizando scikit-learn:** Construir y ajustar un modelo de clasificación binaria robusto utilizando la librería scikit-learn, garantizando métricas operativas de precisión congruentes con el problema de negocio planteado.
- **Serializar el modelo para su uso en producción:** Exportar y empaquetar el estado interno y los pesos del modelo ajustado mediante herramientas de serialización de alto rendimiento como joblib, garantizando una carga rápida libre de efectos colaterales en el entorno servidor.
- **Construir una API REST utilizando FastAPI:** Diseñar y programar una interfaz de red asíncrona de alto rendimiento empleando el framework moderno FastAPI, definiendo rutas estructuradas, documentación OpenAPI interactiva y validaciones nativas de datos.
- **Contenerizar la aplicación mediante Docker:** Empaquetar e inmunizar la base de código, las dependencias del sistema operativo y las librerías de Python en una imagen ligera y segura mediante Docker, eliminando la disparidad de configuraciones de hardware locales.

- **Garantizar un diseño limpio y mantenible:** Adoptar patrones de diseño de software limpio y separación de responsabilidades, aislando el código de conectividad de red de la lógica de inferencia matemática, permitiendo el mantenimiento ágil a largo plazo.

4. Arquitectura del sistema

El diseño arquitectónico adoptado para esta solución sigue un patrón de arquitectura cliente-servidor estrictamente desacoplado. Cada componente cuenta con límites de responsabilidad bien definidos, lo que permite modificar el algoritmo de machine learning subyacente sin alterar los canales de comunicación de red ni los contratos establecidos con sistemas externos.

A continuación, se describen de manera pormenorizada los componentes principales que integran el ecosistema del sistema:

- **Cliente:** Cualquier aplicación o frontend externo (paneles web, aplicaciones móviles, disparadores cron o sistemas legados ERP) que requiera consumir una predicción. Su responsabilidad exclusiva consiste en estructurar y transmitir peticiones HTTP con datos de entrada válidos.
- **Backend (API):** El núcleo lógico del sistema, desarrollado sobre FastAPI. Este componente asume la carga operativa de recibir la petición en crudo, validar la integridad del JSON de entrada, mapear los tipos de datos a variables nativas, orquestar la invocación al pipeline de inferencia y empaquetar la respuesta final.
- **Modelo de Machine Learning:** El binario estático del modelo de machine learning, entrenado previamente de forma offline. Este artefacto es cargado en la memoria RAM del servidor en el instante preciso del arranque de la API (fase de startup), optimizando la velocidad de respuesta al evitar lecturas repetitivas al almacenamiento en disco.
- **Contenedor Docker:** La unidad de aislamiento de infraestructura que encapsula la base de código de la API, el intérprete de Python, las dependencias y las variables de entorno, permitiendo un despliegue idéntico y predecible en cualquier proveedor de nube.

5. Diseño de la API

La interfaz se adhiere a las convenciones de las APIs RESTful modernas, empleando el método HTTP POST para proteger la confidencialidad de los datos analizados dentro del cuerpo de la petición (payload), en lugar de exponerlos en los logs de la URL del servidor.

Endpoint principal de inferencia

Ruta y Método: POST /predict

Entrada (JSON): Carga útil JSON enviada por el cliente de red. Debe cumplir estrictamente con los nombres de atributos y restricciones de rangos numéricos:

```
{
```

```
  "edad": 35,
  "ingresos": 30000,
  "num_transacciones": 120
}
```

Salida (JSON): Estructura de respuesta JSON devuelta de forma determinista por el servidor tras la inferencia exitosa:

```
{
  "prediccion": 0,
  "probabilidad": 0.87
}
```

Validación de datos en tiempo de ejecución

Para garantizar la robustez del servicio ante peticiones malformadas o ataques de denegación de servicio lógico, se utilizan modelos de Pydantic. Este enfoque de validación automática asegura tres niveles críticos de protección de datos en la frontera de la API:

- **Tipos correctos:** Verificación estricta de que los valores recibidos corresponden con las tipificaciones matemáticas requeridas por el modelo scikit-learn (por ejemplo, evitar strings en atributos numéricos).
- **Campos obligatorios:** Intercepción inmediata de payloads incompletos antes de enviarlos al modelo, previniendo excepciones de atributos ausentes (KeyError) dentro de la capa lógica.
- **Estructura consistente:** Garantía de que no se inyecten propiedades espurias o campos maliciosos dentro de las estructuras de control internas del backend.

6. Modelo de datos

En esta arquitectura específica de microservicio de inferencia de alto rendimiento, el modelo de datos es directo y optimizado para CPU, ya que el sistema opera de manera agnóstica al estado interno del usuario, eliminando la sobrecarga y latencia asociadas a la persistencia compleja en bases de datos tradicionales en caliente.

No obstante, el flujo de datos se modela con rigurosidad matemática dividiéndose en dos niveles lógicos claramente diferenciados:

A. Datos de entrada (Input Data Contract)

Representan el conjunto estructurado de características independientes (features) extraídas del dominio del negocio que alimentan la matriz bidimensional del modelo matemático. Cada atributo está acotado por límites lógicos realistas:

- **edad:** Edad biológica verificada del sujeto (Rango aceptado: >= 18 y <= 100 años).
- **ingresos:** Ingresos anuales netos declarados en formato flotante de precisión simple (Rango aceptado: >= 0.0).
- **num_transacciones:** Volumen histórico total de interacciones o transacciones registradas por el usuario en la plataforma (Rango aceptado: >= 0).

B. Datos derivados (Output Data Contract)

Constituyen la carga útil computada que resulta de la transformación del vector de entrada a través de las fronteras de decisión del clasificador:

- **predicción (clase):** Valor entero discreto (0 o 1) que codifica la etiqueta final de predicción asignada por el algoritmo.
- **probabilidad:** Valor continuo flotante entre 0.0 y 1.0 que indica el nivel de confianza o certidumbre estadística que posee el modelo sobre la pertenencia del registro analizado a la clase positiva.

7. Flujo del sistema

El ciclo de vida de procesamiento transaccional de una solicitud sigue un camino lineal determinista. Cada paso del flujo está diseñado para fallar rápido (fail-fast), liberando hilos de ejecución de red si el cliente comete un error en el contrato de datos. La secuencia exacta de operaciones se describe a continuación:

- **1. Solicitud HTTP:** El cliente remoto inicia una conexión TCP/IP e inyecta una petición estructurada HTTP POST hacia la URI '/predict' incluyendo el cuerpo JSON.
- **2. Recepción:** El servidor web ASGI (Uvicorn) intercepta los sockets de red y delega el flujo al kernel de FastAPI de forma asíncrona.
- **3. Validación:** La capa de Pydantic intercepta la trama de bytes de manera automática, realiza el parseo a diccionario de Python y ejecuta las validaciones numéricas restrictivas.
- **4. Transformación y Preprocesado:** Si la estructura es íntegra, el módulo de orquestación extrae los valores numéricos y los transforma en una matriz compatible con los arrays multidimensionales exigidos por las librerías nativas.
- **5. Evaluación del Modelo:** Se invocan los métodos optimizados de inferencia sobre el objeto del modelo que reside permanentemente en la memoria caché del servidor web.
- **6. Inferencia:** El algoritmo matemático de clasificación procesa la matriz de características y genera de manera secuencial la etiqueta de clase y el score probabilístico.
- **7. Estructuración:** Los datos resultantes se inyectan en el constructor del esquema de salida, sanitizando los formatos numéricos y garantizando la homogeneidad estructural.
- **8. Respuesta de Red:** La API serializa el esquema a formato de texto plano JSON y devuelve una respuesta con código de estado HTTP 200 OK hacia la interfaz de red del cliente.

8. Integración del modelo

La integración industrial del modelo se rige por el principio de segregación total de entornos. El ciclo de vida de entrenamiento conceptual del algoritmo ocurre de forma externa y asíncrona, desacoplado del código del servidor de la API, asegurando que la computación pesada de gradientes u optimización de hiperparámetros no interfiera con el tiempo de respuesta de red.

Fase de Entrenamiento (Script de Data Science Offline)

Llevado a cabo por el equipo de ciencia de datos empleando algoritmos robustos de ensamble sobre scikit-learn. El script base de generación del artefacto matemático sigue la siguiente estructura limpia de inicialización:

```
from sklearn.ensemble import RandomForestClassifier
import numpy as np

# Simulación de datos de entrenamiento históricos consolidados
X_train = np.array([[30, 50000, 100], [45, 80000, 250], [22, 15000, 10], [60, 120000,
400]])
y_train = np.array([0, 1, 0, 1])

# Instanciación del estimador con hiperparámetros bloqueados para reproducibilidad
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

Fase de Serialización y Volcado Estático

Una vez que las métricas de validación cruzada alcanzan los umbrales exigidos por la dirección de operaciones, se procede al volcado atómico del estado del objeto de memoria hacia un archivo binario inmutable mediante el uso de la librería joblib:

```
import joblib
import os

# Persistencia persistente en el sistema de archivos del proyecto
output_dir = "models"
os.makedirs(output_dir, exist_ok=True)
model_filename = os.path.join(output_dir, "model.pkl")
joblib.dump(model, model_filename)
print(f"Artefacto matemático serializado con éxito en: {model_filename}")
```

Fase de Carga en Memoria RAM (API Gateway)

Durante la inicialización del proceso de la API web, el backend lee una única vez el binario y mantiene el puntero cargado en el heap de memoria, minimizando drásticamente la latencia de Entrada/Salida (I/O):

```
import joblib
import os

MODEL_PATH = os.path.join(os.path.dirname(__file__), "../models/model.pkl")
if not os.path.exists(MODEL_PATH):
    raise FileNotFoundError(f"Fallo crítico de inicialización: Archivo {MODEL_PATH} no
detectado")

loaded_model = joblib.load(MODEL_PATH)
```

8.1 Implementación Completa del Código Fuente del Sistema

A fin de eliminar la ambigüedad y cumplir estrictamente con los estándares de desarrollo profesionales, a continuación se inyecta la base de código completa, modularizada y libre de omisiones de los tres componentes medulares que configuran el paquete `/app`.

Módulo de Esquemas y Contratos de Validación (app/schemas.py):Código completo del archivo encargado del contrato de entrada y salida, asegurando límites numéricos y tipado estricto:

```
from pydantic import BaseModel, Field, field_validator

class PredictionInput(BaseModel):
    edad: int = Field(..., ge=18, le=100, description="Edad biológica del usuario
solicitante", example=35)
    ingresos: float = Field(..., ge=0.0, description="Ingresos anuales netos
declarados", example=30000.0)
    num_transacciones: int = Field(..., ge=0, description="Volumen total de
transacciones históricas", example=120)

    @field_validator('edad')
    @classmethod
    def validar_limite_edad(cls, value: int) -> int:
        if value < 18 or value > 100:
            raise ValueError("La edad proporcionada se encuentra fuera de los límites
operativos autorizados (18-100).")
        return value

class PredictionOutput(BaseModel):
    prediccion: int = Field(..., description="Clase binaria asignada por el modelo (0
o 1)")
```

```
    probabilidad: float = Field(..., description="Probabilidad matemática de
pertenencia a la clase positiva")
    status: str = Field("SUCCESS", description="Estado operativo de la inferencia
ejecutada")
```

Módulo de Abstracción del Modelo Predictivo (app/model.py):Código encargado de gestionar el estado del modelo matemático, aplicando el patrón Wrapper para la carga atómica:

```
import joblib
import os
import numpy as np
from app.schemas import PredictionInput

class ModelManager:
    def __init__(self):
        self.model = None
        self.model_path = os.path.join(os.path.dirname(__file__),
"../models/model.pkl")

    def load_model_artifact(self):
        if not os.path.exists(self.model_path):
            raise FileNotFoundError(f"Error de Infraestructura: Binario no hallado en
{self.model_path}")
        self.model = joblib.load(self.model_path)

    def execute_inference(self, data: PredictionInput) -> tuple:
        if self.model is None:
            raise RuntimeError("Error Operacional: El modelo no ha sido cargado
explícitamente en el arranque.")

        # Transformación estructural a matriz bidimensional de tipo NumPy exigida por
Scikit-Learn
        input_features = np.array([[
            data.edad,
            data.ingresos,
            data.num_transacciones
        ]])

        # Cómputo de predicción y probabilidades analíticas
        prediction_class = int(self.model.predict(input_features)[0])
        probabilities = self.model.predict_proba(input_features)[0]
        prediction_probability = float(probabilities[1]) if len(probabilities) > 1
else float(probabilities[0])

        return prediction_class, prediction_probability

model_orchestrator = ModelManager()
```

Módulo del Servidor REST Central (app/main.py): Punto de entrada unificado que orquestal el enrutamiento HTTP y gestiona los ciclos de vida mediante hooks asíncronos:

```
from fastapi import FastAPI, HTTPException, status
from app.schemas import PredictionInput, PredictionOutput
from app.model import model_orchestrator
from contextlib import asynccontextmanager
import logging

# Configuración del sistema corporativo de trazabilidad y logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -
%(message)s')
logger = logging.getLogger("ProductionAPI")

@asynccontextmanager
async def application_lifespan(app_instance: FastAPI):
    # Evento de inicialización atómica en el arranque del hardware
    try:
        model_orchestrator.load_model_artifact()
        logger.info("INFRAESTRUCTURA: Binario del modelo de Machine Learning montado
con éxito en RAM.")
    except Exception as exc:
        logger.critical(f"FALLO CRÍTICO DE STARTUP: No se pudo instanciar el modelo:
{str(exc)}")
        raise exc
    yield
    # Evento de apagado limpio del proceso servidor
    logger.info("INFRAESTRUCTURA: Liberando y limpiando recursos de la API REST.")

app = FastAPI(
    title="API de Predicción Corporativa - ML Clásico",
    version="1.0.0",
    description="Servicio de alto rendimiento para la evaluación automatizada mediante
modelos serializados.",
    lifespan=application_lifespan
)

@app.get("/health", status_code=status.HTTP_200_OK, tags=["Monitoreo"])
def perform_health_check():
    is_active = model_orchestrator.model is not None
    return {
        "status": "healthy" if is_active else "unhealthy",
        "model_loaded": is_active
    }

@app.post("/predict", response_model=PredictionOutput, status_code=status.HTTP_200_OK,
tags=["Inferencia"])
def predict(payload: PredictionInput):
    try:
        pred_class, pred_prob = model_orchestrator.execute_inference(payload)
        logger.info(f"INFERENCIA EXITOSA: Predicción={pred_class} |
```

```
Probabilidad={pred_prob:.4f}")
        return PredictionOutput(prediccion=pred_class, probabilidad=pred_prob)
    except Exception as error:
        logger.error(f"ERROR OPERATIVO INTERNO: {str(error)}")
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Fallo interno en el motor matemático de predicción: {str(error)}"
        )
```

9. Despliegue y Docker

Con la finalidad primordial de certificar el principio de inmutabilidad en la capa de operaciones y neutralizar de forma absoluta el clásico problema en la ingeniería de sistemas: 'en la máquina del desarrollador sí funciona', la infraestructura de este proyecto se define completamente como código utilizando contenedores Docker estandarizados.

Archivo de Configuración del Entorno de Contenedor (Dockerfile)

Se selecciona una imagen base oficial de Python basada en una distribución optimizada y ultra-ligera (slim-buster), la cual minimiza drásticamente la superficie de vulnerabilidades de seguridad y acelera la velocidad de descarga en entornos de integración continua (CI/CD):

```
FROM python:3.10-slim-buster

# Bloqueo de variables de entorno para optimizar la ejecución de Python en capas
aisladas
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=off \
    PIP_DISABLE_PIP_VERSION_CHECK=on

WORKDIR /app

# Instalación de dependencias críticas del sistema si fuesen requeridas
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Copia aislada del manifiesto de librerías para explotar el almacenamiento en caché
de Docker
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt

# Copia integral de los módulos de software y binarios matemáticos estructurados
COPY ./app ./app
COPY ./models ./models
```

```
# Creación y asignación de un usuario de seguridad sin privilegios de root
RUN useradd -u 8888 appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000

# Comando de arranque optimizado utilizando 4 workers concurrentes bajo protocolo ASGI
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers",
"4"]
```

Orquestación Multi-Servicio Opcional (docker-compose.yml)

Permite simplificar de forma sustancial el despliegue local o en servidores staging, aislando las redes virtuales de comunicación y preparando el entorno para acoplar servicios complementarios como proxys inversos o bases de datos relacionales en fases futuras del desarrollo:

```
version: '3.8'

services:
  prediction-api:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: production_prediction_service
    ports:
      - "8000:8000"
    environment:
      - LOG_LEVEL=info
    restart: always
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
```

10. Estructura del proyecto

Para mantener una estricta modularidad y cumplir rigurosamente con los patrones de diseño de software limpio (Clean Architecture), el repositorio organiza sus archivos de forma jerárquica, separando de manera inequívoca las definiciones estáticas de los algoritmos matemáticos de los canales de comunicación HTTP. A continuación, se detalla la topología exacta del árbol de directorios del sistema:

```
/
├── app/
│   ├── __init__.py      # Inicializador de paquete que permite importaciones
relativas limpias
│   ├── main.py          # Punto de entrada de FastAPI, enrutamiento y ciclo de vida
(lifespan)
│   ├── model.py         # Orquestador y Wrapper de inferencia para Scikit-Learn y
NumPy
│   └── schemas.py       # Modelos Pydantic para la validación estricta de contratos
de datos
├── models/
│   └── model.pkl        # Artefacto binario serializado que contiene los pesos e
hiperparámetros
├── Dockerfile           # Receta de construcción automatizada e inmutable de la
imagen de producción
├── docker-compose.yml   # Orquestador del stack de servicios y configuración de redes
puente
└── requirements.txt     # Manifiesto estricto de dependencias y versiones bloqueadas
del proyecto
```

11. Buenas prácticas

El desarrollo de software de nivel industrial exige la adopción dogmática de directrices que blinden el comportamiento del sistema ante condiciones extremas de tráfico u obsolescencia tecnológica. En este microservicio se han integrado las siguientes directrices críticas:

- **Separar de forma estricta la lógica de negocio y la API:** La API actúa exclusivamente como una pasarela conductora. Jamás debe inyectarse lógica de preprocesamiento matemático complejo, cálculos estadísticos pesados o manipulación de arrays directamente dentro de las funciones controladoras de los endpoints REST en main.py.
- **Evitar código acoplado en endpoints:** Cualquier interacción con librerías pesadas de procesamiento de datos debe ser delegada a clases abstractas independientes dentro del archivo model.py, aislando el framework web del motor de inteligencia artificial.
- **Manejar errores de forma correcta y controlada:** El código cuenta con capturas explícitas de excepciones genéricas e internas del motor matemático, traduciéndolas automáticamente en códigos de error estandarizados del protocolo HTTP (como 422 Unprocessable Entity o 500 Internal Server Error) acompañados de payloads JSON explicativos para el cliente, evitando fugas de stack traces del sistema operativo.
- **Registrar trazas de actividad (logs):** Se implementa un motor estructurado de logs integrado en los hilos del servidor que registra cronológicamente cada inferencia exitosa, las probabilidades calculadas y las anomalías de hardware detectadas, habilitando auditorías funcionales posteriores.
- **Validar los datos de entrada (inputs) de forma perimetral:** Se rechaza activamente cualquier paquete que no cumpla de forma exacta con los contratos numéricos preestablecidos en el esquema Pydantic antes de consumir ciclos valiosos del procesador.

12. Escalabilidad

El diseño arquitectónico de este microservicio no es estático; ha sido concebido desde sus cimientos para crecer e integrarse de manera fluida en arquitecturas distribuidas enterprise de gran envergadura. A medida que las demandas de tráfico transaccional se multipliquen, el sistema se encuentra preparado para evolucionar hacia los siguientes estadios de madurez de infraestructura:

- **Microservicio independiente de alta disponibilidad:** El contenedor stateless resultante puede ser clonado indefinidamente detrás de un balanceador de carga de red (como Nginx o AWS ALB), distribuyendo el tráfico HTTP de forma homogénea.
- **Integración asíncrona mediante colas de mensajería:** Para escenarios donde la inferencia no requiera sincronía inmediata (procesamiento por lotes de millones de registros de Churn), la API puede transformarse en un consumidor asíncrono acoplado a brókers de mensajería masiva como RabbitMQ o Apache Kafka.
- **Despliegue y orquestación elástica en Kubernetes:** Habilita el despliegue del contenedor dentro de pods orquestados, configurando políticas de autoescalado horizontal automatizado (Horizontal Pod Autoscaler - HPA) que instancian dinámicamente nuevas réplicas del backend en función de la degradación real del uso de CPU o memoria RAM en el clúster.
- **Monitorización avanzada con Prometheus:** Integración nativa con agentes de recolección de métricas como Prometheus e interfaces de visualización en tiempo real como Grafana, exponiendo los tiempos de latencia analítica en el percentil 99 (p99) y la tasa de códigos de error HTTP de red.

13. Conclusión

La implementación exhaustiva de este sistema técnico demuestra de manera fehaciente que la puesta en producción de modelos de Machine Learning requiere una convergencia equilibrada e interdisciplinar entre la ciencia de datos y la ingeniería de software tradicional de nivel enterprise.

A través del uso estructurado de FastAPI, la inmutabilidad garantizada por Docker y el aislamiento lógico de capas con Pydantic, este proyecto constituye un plano de arquitectura definitivo y un artefacto base robusto para desplegar servicios de inteligencia artificial estables, seguros y escalables en entornos industriales reales, consolidando habilidades críticas de nivel backend e ingeniería de IA aplicada.